Python-Based Testing & Validation for STM32 Synchronous Buck Power Supply

Variable Switching DC Power Supply | 1.25–30 V | 0–5 A | >88% Efficiency

This document provides a comprehensive guide to using Python for automated test and validation of the STM32-based synchronous buck supply. It covers instrument control, serial telemetry, automated test scripts, regression frameworks, and report generation.

1. The Python Test Stack

Layer	Library	Purpose
Instrument Control	PyVISA	Talk to scopes, DMMs, power supplies, e-loads via GPIB/USB/LAN
Serial / Debug	pyserial	Read telemetry from STM32 (PA9 USART1_TX)
Data & Math	pandas, numpy	Store measurements, compute efficiency, statistics
Visualization	matplotlib	Plot efficiency curves, transient waveforms, ripple
Reporting	openpyxl / python-docx	Auto-generate test reports with tables and plots
Test Framework	pytest (optional)	Structured pass/fail assertions for regression testing

Install everything with:

Note: pyvisa-py is the pure-Python VISA backend — no NI drivers needed for basic USBTMC or serial instruments.

```
pip install pyvisa pyserial pyvisa-py pandas
numpy matplotlib openpyxl
```

2. Connecting to Your Instruments

Finding Your Instruments

```
import pyvisa

rm = pyvisa.ResourceManager()
print(rm.list_resources())
# Output example:
('USB0::0x1AB1::0x0588::DS1ZA123456789::INSTR',
...)
```

Generic Instrument Class

```
class BenchInstrument:
    def __init__(self, resource_string):
        self.rm = pyvisa.ResourceManager()
        self.inst =
self.rm.open_resource(resource_string)
        self.inst.timeout = 5000  # ms

    def write(self, cmd):
        self.inst.write(cmd)

    def query(self, cmd):
        return self.inst.query(cmd).strip()

    def idn(self):
        return self.query("*IDN?")

    def reset(self):
        self.write("*RST")
```

3. Specific Instrument Wrappers

Programmable DC Power Supply (e.g., Rigol DP832)

```
class DCPowerSupply(BenchInstrument):
    def set_voltage(self, channel, voltage):
        self.write(f"SOURCE{channel}:VOLTAGE
{voltage}")

    def set_current(self, channel, current):
        self.write(f"SOURCE{channel}:CURRENT
{current}")

    def output_on(self, channel):
        self.write(f"OUTPUT{channel} ON")

    def output_off(self, channel):
        self.write(f"OUTPUT{channel} OFF")

    def measure_voltage(self, channel):
        return
float(self.query(f"MEASURE:VOLTAGE?
CH{channel}"))

    def measure_current(self, channel):
        return
float(self.query(f"MEASURE:CURRENT?
CH{channel}"))
```

Electronic Load (e.g., Keysight/Agilent)

```
class ElectronicLoad(BenchInstrument):
    def set_mode(self, mode="CURRENT"):
        self.write(f"MODE {mode}")  # CURRENT,
VOLTAGE, RESISTANCE, POWER

    def set_current(self, current):
```

```
        self.write(f"CURRENT {current}")

    def input_on(self):
        self.write("INPUT ON")

    def input_off(self):
        self.write("INPUT OFF")

    def measure_voltage(self):
        return
float(self.query("MEASURE:VOLTAGE?"))

    def measure_current(self):
        return
float(self.query("MEASURE:CURRENT?"))
```

Digital Multimeter (e.g., Keysight 34461A)

```
  class DMM(BenchInstrument):
      def configure_dc_voltage(self, range_val=10):
          self.write(f"CONF:VOLT:DC {range_val}")

      def read(self):
          return float(self.query("READ?"))

      def configure_4wire_resistance(self):
          self.write("CONF:FRES")  # 4-wire for
  accurate shunt measurement
```

Oscilloscope (e.g., Rigol DS1054Z)

```
class Oscilloscope(BenchInstrument):
    def autoscale(self):
        self.write(":AUTOSCALE")

    def set_timebase(self, scale):
        self.write(f":TIMEBASE:SCALE {scale}")
```

```
    def set_channel_scale(self, ch, scale):
        self.write(f":CHANNEL{ch}:SCALE {scale}")

    def measure_vpp(self, ch):
        return float(self.query(f":MEASURE:VPP?
CHANNEL{ch}"))

    def measure_frequency(self, ch):
        return
float(self.query(f":MEASURE:FREQUENCY?
CHANNEL{ch}"))

    def run(self):
        self.write(":RUN")

    def stop(self):
        self.write(":STOP")

    def single(self):
        self.write(":SINGLE")

    def get_waveform(self, ch):
        self.write(f":WAVEFORM:SOURCE CHANNEL{ch}")
        self.write(":WAVEFORM:MODE NORMAL")
        self.write(":WAVEFORM:FORMAT BYTE")
        raw =
self.inst.query_binary_values(":WAVEFORM:DATA?",
datatype='B')
        return raw
```

4. STM32 Serial Telemetry Reader

Your STM32 outputs debug telemetry on PA9 (USART1_TX). You can read this in real-time to monitor internal states (ADC raw counts, PID duty cycle, fault flags) while the Python test runs.

```
import serial
import threading
import queue

class STM32Telemetry:
    def __init__(self, port='/dev/ttyUSB0',
baudrate=115200):
        self.ser = serial.Serial(port, baudrate,
timeout=1)
        self.data_queue = queue.Queue()
        self.running = False
        self.latest = {}

    def start(self):
        self.running = True
        self.thread =
threading.Thread(target=self._read_loop)
        self.thread.daemon = True
        self.thread.start()

    def _read_loop(self):
        while self.running:
            try:
                line =
self.ser.readline().decode('utf-8').strip()
                if line:
                    # Expect CSV:
Vset,Vmeas,Imeas,Duty,Temp,Status
                    parts = line.split(',')
                    if len(parts) == 6:
                        self.latest = {
                            'v_set':
float(parts[0]),
                            'v_meas':
float(parts[1]),
```

```
                                'i_meas':
  float(parts[2]),
                                'duty':
  float(parts[3]),
                                'temp':
  float(parts[4]),
                                'status': parts[5]
                            }

  self.data_queue.put(self.latest)
              except Exception as e:
                  print(f"Serial error: {e}")

      def get_latest(self):
          return self.latest

      def stop(self):
          self.running = False
          self.ser.close()
```

STM32 Firmware Side (add to your main.c):

```
// Call this in your main loop or at 10 Hz from a
timer
void SendTelemetry(void) {
    printf("%.3f,%.3f,%.3f,%.2f,%.1f,%s\r\n",
        v_setpoint,
        ADC_ReadVoltage(),
        INA226_ReadCurrent(),
        duty_cycle,
        ADC_ReadTemp(),
        (fault_flag ? "FAULT" : "OK")
    );
}
```

5. Complete Automated Test: Load Regulation

This script performs the Section 5 Load Regulation test automatically — fix output at 12 V, sweep load 0 -> 5 A in 0.5 A steps, measure Vout with DMM, log everything, and generate plots.

```
import time
import pandas as pd
import matplotlib.pyplot as plt

def test_load_regulation(psu_resource,
load_resource, dmm_resource,
                         stm32_port=None,
output_csv="load_reg_results.csv"):
    # Initialize instruments
    psu = DCPowerSupply(psu_resource)
    load = ElectronicLoad(load_resource)
    dmm = DMM(dmm_resource)

    telemetry = None
    if stm32_port:
        telemetry = STM32Telemetry(stm32_port)
        telemetry.start()

    results = []

    try:
        psu.reset()
        load.reset()
        dmm.configure_dc_voltage(range_val=20)

        # Set bench supply to 45 V (simulating
rectified DC bus)
        psu.set_voltage(1, 45.0)
        psu.set_current(1, 2.0)  # 2A limit for
safety
```

```
        psu.output_on(1)
        load.set_mode("CURRENT")
        load.input_off()

        print("Starting load regulation sweep...")

        for current in [0, 0.5, 1.0, 1.5, 2.0, 2.5,
3.0, 3.5, 4.0, 4.5, 5.0]:
            load.set_current(current)
            load.input_on()
            time.sleep(0.5)

            vout_dmm = dmm.read()
            vin = psu.measure_voltage(1)
            iin = psu.measure_current(1)
            pin = vin * iin
            vout_load = load.measure_voltage()
            iout_load = load.measure_current()
            pout = vout_load * iout_load
            efficiency = (pout / pin * 100) if pin
> 0 else 0
            telem = telemetry.get_latest() if
telemetry else {}

            row = {
                'timestamp':
time.strftime('%H:%M:%S'),
                'load_current_set': current,
                'vout_dmm': vout_dmm,
                'vout_load': vout_load,
                'iout_load': iout_load,
                'vin': vin, 'iin': iin, 'pin': pin,
'pout': pout,
                'efficiency': efficiency,
                'v_set_stm32': telem.get('v_set'),
```

```
                'v_meas_stm32':
telem.get('v_meas'),
                'duty_stm32': telem.get('duty'),
            }
            results.append(row)

            print(f"Iload={current:.1f}A |
Vout={vout_dmm:.3f}V | "
                  f"Eff={efficiency:.1f}% |
Duty={telem.get('duty', 'N/A')}")

            if abs(vout_dmm - 12.0) > 12.0 * 0.005:
                print(f"  ⚠️  FAIL: Regulation out
of spec at {current}A")

        df = pd.DataFrame(results)
        df.to_csv(output_csv, index=False)
        print(f"\n✅ Results saved to
{output_csv}")

        # Plot
        fig, (ax1, ax2) = plt.subplots(2, 1,
figsize=(10, 8))
        ax1.plot(df['load_current_set'],
df['vout_dmm'], 'b-o', label='Vout (DMM)')
        ax1.axhline(y=12.0, color='r', linestyle='-
-', label='Target 12V')
        ax1.axhline(y=11.94, color='g',
linestyle=':', label='±0.5% limit')
        ax1.axhline(y=12.06, color='g',
linestyle=':')
        ax1.set_xlabel('Load Current (A)')
        ax1.set_ylabel('Output Voltage (V)')
        ax1.set_title('Load Regulation @ 12V')
        ax1.legend(); ax1.grid(True)
```

```
        ax2.plot(df['load_current_set'],
df['efficiency'], 'r-s', label='Efficiency')
        ax2.axhline(y=88, color='k', linestyle='--
', label='Target 88%')
        ax2.set_xlabel('Load Current (A)')
        ax2.set_ylabel('Efficiency (%)')
        ax2.set_title('Efficiency vs. Load')
        ax2.legend(); ax2.grid(True)

        plt.tight_layout()
        plt.savefig('load_regulation_plot.png',
dpi=150)
        plt.show()
        return df

    finally:
        load.input_off()
        psu.output_off(1)
        if telemetry:
            telemetry.stop()
        print("Instruments safely shut down.")

# Run it
if __name__ == "__main__":
    df = test_load_regulation(

psu_resource="USB0::0x1AB1::0x0E11::DP8C1234567::IN
STR",
        load_resource="GPIB0::5::INSTR",

dmm_resource="USB0::0x2A8D::0x1301::MY12345678::INS
TR",
        stm32_port="/dev/ttyUSB0"
    )
```

6. Automated Transient Response Capture

For the Section 5B Load Transient test, trigger the scope on the load step and capture voltage deviation.

```
def test_load_transient(scope_resource,
load_resource,
                        initial_current=0,
final_current=5.0,

output_png="transient_response.png"):
    scope = Oscilloscope(scope_resource)
    load = ElectronicLoad(load_resource)

    scope.reset()
    scope.set_timebase(100e-6)      # 100 µs/div
    scope.set_channel_scale(1, 0.5) # 0.5 V/div for
Vout (AC coupled)
    scope.set_channel_scale(2, 2)  # 2 V/div for
load current

    # Trigger on Vout deviation (falling edge)
    scope.write(":TRIGGER:EDGE:SOURCE CHANNEL1")
    scope.write(":TRIGGER:EDGE:SLOPE FALL")
    scope.write(":TRIGGER:EDGE:LEVEL 11.5")
    scope.single()

    load.set_mode("CURRENT")
    load.set_current(initial_current)
    load.input_on()
    time.sleep(0.5)

    # Step load
    load.set_current(final_current)
    time.sleep(0.1)
```

```
    scope.stop()
    vout_wave = scope.get_waveform(1)

    import numpy as np
    vout = np.array(vout_wave)
    time_axis = np.linspace(0, len(vout) * 100e-6,
len(vout))

    plt.figure(figsize=(12, 5))
    plt.plot(time_axis * 1e3, vout)
    plt.axvline(x=0, color='r', linestyle='--',
label='Load Step')
    plt.xlabel('Time (ms)')
    plt.ylabel('Output Voltage (raw)')
    plt.title(f'Load Transient: {initial_current}A
→ {final_current}A')
    plt.grid(True); plt.legend()
    plt.savefig(output_png); plt.show()
    load.input_off()
```

7. Automated Protection Testing

For Section 9 Protection Tests, script fault injection and measure response time.

```
import time

def test_ovp_response(scope_resource, stm32_port,
trip_voltage=32.0):
    scope = Oscilloscope(scope_resource)
    telem = STM32Telemetry(stm32_port)
    telem.start()

    scope.set_timebase(10e-6)  # 10 µs/div for fast
events
    scope.single()
```

```
    # Trigger fault: send command to STM32 to set
35 V
    telem.ser.write(b"SETV 35.0\n")
    time.sleep(0.1)

    latest = telem.get_latest()
    if latest.get('status') == 'FAULT':
        print(f"✅ OVP triggered. Vout dropped to
~0V.")
    else:
        print("❌ OVP did not trigger!")

    telem.stop()
```

8. Full Regression Test Suite (pytest)

Wrap everything in a pytest framework for automated regression testing.

```
# test_power_supply.py
import pytest

class TestPowerSupply:
    @pytest.fixture(scope="class")
    def instruments(self):
        psu = DCPowerSupply("USB0::...::INSTR")
        load = ElectronicLoad("GPIB0::5::INSTR")
        dmm = DMM("USB0::...::INSTR")
        yield {'psu': psu, 'load': load, 'dmm':
dmm}
        load.input_off()
        psu.output_off(1)

    def test_voltage_accuracy_12V(self,
instruments):
        instruments['psu'].output_on(1)
```

```
        time.sleep(0.5)
        vout = instruments['dmm'].read()
        assert abs(vout - 12.0) < 0.06, f"12V
accuracy failed: {vout}V"

    def test_load_regulation_5A(self, instruments):
        instruments['load'].set_current(5.0)
        instruments['load'].input_on()
        time.sleep(0.5)
        vout = instruments['dmm'].read()
        assert abs(vout - 12.0) < 0.06, f"Load
regulation failed at 5A: {vout}V"

    def test_efficiency_full_load(self,
instruments):
        # measure and assert > 88%
        pass

# Run: pytest test_power_supply.py -v
```

9. Data Logging & Report Generation

Automatically generate a Word report with plots and tables from test data.

```
from docx import Document
from docx.shared import Inches

def generate_report(df, plot_path,
output_doc="Test_Report.docx"):
    doc = Document()
    doc.add_heading('Power Supply Test Report', 0)

    table = doc.add_table(rows=1, cols=3)
    table.style = 'Table Grid'
    hdr = table.rows[0].cells
    hdr[0].text = 'Parameter'
```

```
    hdr[1].text = 'Spec'
    hdr[2].text = 'Measured'

    max_eff = df['efficiency'].max()
    min_v = df['vout_dmm'].min()

    rows = [
        ['Max Efficiency', '> 88%',
f'{max_eff:.1f}%'],
        ['Min Vout @ 5A', '11.94 V', f'{min_v:.3f}
V'],
        ['Load Regulation', '< ±0.5%',
f'{abs(min_v-12)/12*100:.2f}%'],
    ]
    for r in rows:
        cells = table.add_row().cells
        for i, txt in enumerate(r):
            cells[i].text = txt

    doc.add_paragraph()
    doc.add_picture(plot_path, width=Inches(5.5))
    doc.save(output_doc)
    print(f"Report saved: {output_doc}")
```

10. Best Practices for Python Test Automation

Practice	Why It Matters
Always use try/finally	Guarantees instruments shut down safely even if a test crashes mid-run
Log raw instrument readings	If a test fails, you can debug without re-running
Use pandas for data, not lists	Built-in CSV export, statistics, plotting
Version-control your test scripts	Git track test code alongside firmware/hardware revisions
Add human-in-the-loop gates	For dangerous tests (short circuit,

	OVP), require input("Confirm: ")
Timestamp everything	Correlates test data with thermal conditions, component batch, etc.
Separate test config from code	Use JSON/YAML for voltage setpoints, limits — edit without touching Python

Summary: What Python Gives You

Manual Test	Python Automation
Hand-write 11 load points in a notebook	Sweep 0–5 A in 0.1 A steps in 2 minutes
Eyeball ripple on scope	Auto-capture, measure Vpp, compare to 30 mV limit
Calculate efficiency on calculator	Auto-compute Pin/Pout at every point, plot curve
Forget what the DMM read at 3.5 A	Everything logged with timestamps to CSV
Subjective "looks good" pass/fail	Hard assertions: assert efficiency > 88
Hours of repetitive testing	One script, unattended, overnight burn-in

This is exactly the 'automated hardware and software solutions to test+validate electronics hardware' that Astranis describes in the job posting. Implementing the Load Regulation and Efficiency Sweep scripts above provides a compelling demo project for technical interviews.